

Monte Carlo Simulation - Portfolio loss and importance sampling

Thomas Sejer Canham

October 9, 2026

Aarhus University

Contents

1	Introduction	1
2	Crude Monte Carlo Simulation	1
3	Control Variable	2
4	Comparison of the methods	2
5	Appendix	3

1 Introduction

In this project, we will investigate the use of Monte Carlo simulation (MCS) to estimate the exceedance probability $\mathbb{P}(L > x)$ of a loss

$$L = \delta^T \Delta S + \Delta S^T \Gamma \Delta S,$$

where, $\delta \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim \mathcal{N}(0, \Sigma_S)$ is a (multivariate) normal random variable. The exceedance probability is the probability that the loss L exceeds a certain threshold $x = 10$.

In the parts (a), (b) and (c), we will focus on the one-dimensional case, $m = 1$. These three parts will compare crude MCS with MCS where we introduce control variates and importance sampling respectively. In part (d), the differences of these methods are visualized by comparison of the confidence intervals of the estimates that are obtained from each method.

Parameters for the various codes snippets can be seen in the appendix in [Listing 2](#).

2 Crude Monte Carlo Simulation

In this section, we will implement a crude Monte Carlo simulation to estimate the exceedance probability $\mathbb{P}(L > x)$ for the one-dimensional case ($m = 1$). We are interested in $\mathbb{P}(L > x)$. We may introduce the indicator function $\mathbb{I}(L > x)$ of the event $L > x$. If we compute the expectation of the indicator function, we obtain the exceedance probability

$$\mathbb{P}(L > x) = \mathbb{E}[\mathbb{I}(L > x)] = 0 \cdot \mathbb{P}(L > x) + 1 \cdot \mathbb{P}(L > x) = \mathbb{P}(L > x), \quad (1)$$

by the bernoulli property of the indicator function. We estimate the exceedance probability by use of the following Monte Carlo estimator

$$\hat{\ell} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(L_i > x) \xrightarrow[N \rightarrow \infty]{a.s.} \mathbb{E}[\mathbb{I}(L > x)], \quad (2)$$

by the law of large numbers. The crude Monte Carlo simulation is implemented in the code snippet seen in [Listing 1](#). A estimate of

$$\hat{\ell} \approx 0.0103$$

is found with a variance of $s^2 = 1.03 \cdot 10^{-7}$.

Listing 1: Python code for crude MCS

```
1  """
2  Code for task (a) - Crude monte carlo
3
4  """
5
6  from parameters import *
7
8  def indicator(x, L):
9      # Exceedance of x
10     return L > x
11
12  def L(Delta_s, delta, gamma):
13     """L for N simulations
14
15     input:
16     N (int): Number of simulations
17     Delta_S: N(0, Sigma_S) of shape (m, N) samples
18     gamma: vector of size m
19     delta: vector of size m
20     """
21     # Returns (N,) losses
22     L = delta.T @ Delta_s + np.sum(Delta_s * (gamma @ Delta_s), axis=0)
23     return L
24
```

```

25 def crude_monte_carlo(N, Delta_s= None):
26     """
27     Crude Monte Carlo simulation to estimate the probability of exceedance of x.
28
29     Parameters:
30     N (int): Number of simulations
31
32     Returns:
33     float: Estimated probability of exceedance
34     """
35
36     if Delta_s is None:
37         Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size = N).
38         T
39
40     # Simulate N outcomes of L
41     loss = L(Delta_s, delta, gamma)
42
43     # Mean of indicator converges a.s to P(L > x)
44     exceedance = indicator(x, loss)
45     # Estimate
46     exceedance_prop = np.mean(exceedance)
47     # Variance of estimate
48     variance = np.var(exceedance) / N
49     return exceedance_prop, variance
50
51 if __name__ == "__main__":
52     print("Crude Monte Carlo simulation: \n")
53     np.random.seed(0)
54     N = 100000
55     # Estimate and variance of estimator
56     l, s2 = crude_monte_carlo(N)
57     print("Estimated probability of exceedance of x: ", l)
58     print("Variance of MC estimator: ", s2)

```

3 Control Variable

A control variable is now introduced with the aim of reducing the variance of the Monte Carlo estimator. One caveat of the control variable method is that we need a closed form expression for the expectation of the control variable. We introduce the control variable $Y = \Delta S$ and compute its expectation

4 Comparison of the methods

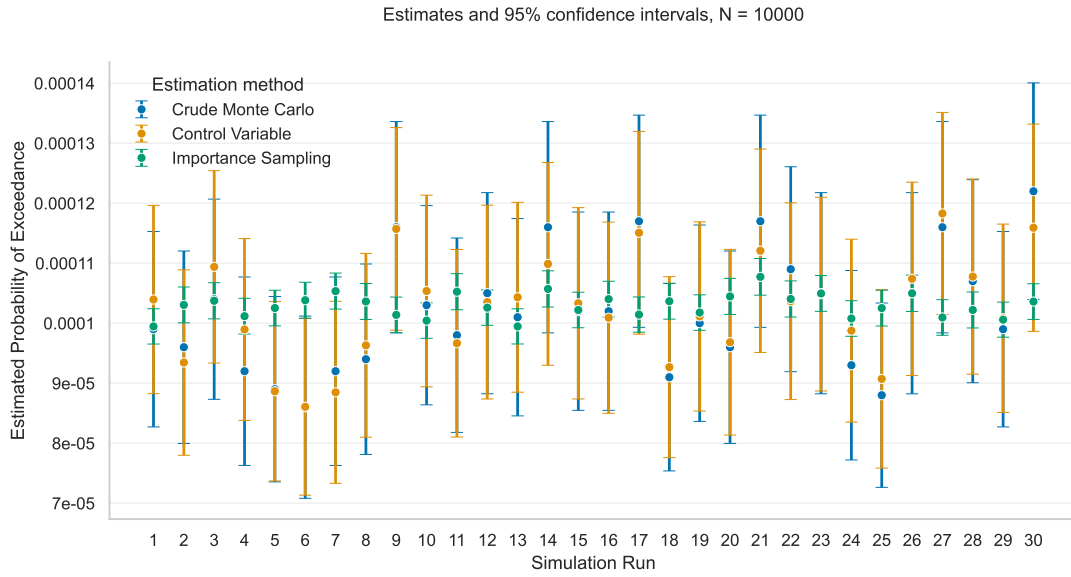


Figure 1: Comparison of confidence intervals for crude MCS, control variable MCS and importance sampling MCS. The confidence intervals are based on 30 simulations with $N = 10^4$ samples each.

5 Appendix

Listing 2: parameters.py - Parameters for the various code snippets

```

1 import numpy as np
2 ### (a), (b) ###
3
4 m = 1
5 # delta, gamma is a column vector of size m
6 delta = np.ones((m, 1)) * 2
7 gamma = np.ones((m,m))
8 x = 10
9 sigma_s = np.ones((m, m))
10
11 ### (c) ###
12
13 # Q measure paramters
14 mu_q = np.ones(m) * 2
15 sigma_q = sigma_s
16
17 # P measure parameters
18 mu_p = np.zeros(m)
19 sigma_p = sigma_s
20
21 ##### (d) #####
22 import seaborn as sns
23 from scipy.stats import norm
24
25 Nd = 10000
26
27 SIMULATION_RUNS = 30
28 plot_n = np.arange(1, SIMULATION_RUNS + 1)
29
30 methods = ["Crude Monte Carlo", "Control Variable", "Importance Sampling"]
31 colors = sns.color_palette("colorblind", 3)
32
33 def confidence_interval(s, l, N, level = 0.05):
34     """Compute confidence interval, equation (1.2)"""
35     z_alpha = norm.ppf(1 - level)

```

```
36 | return (1 - z_alpha*s / np.sqrt(N), 1 + z_alpha*s / np.sqrt(N) )
```